

Code Companion

A line-by-line guide to the csdh-postop-seizure-risk analysis code

Niels Pacheco-Barrios

Department of Neurosurgery, Beth Israel Deaconess Medical Center, Harvard Medical School

May 19, 2026

Each chapter explains one part of the codebase in plain English, then shows the actual code with line numbers and syntax highlighting, and then unpacks what each block is doing. You do not need to be fluent in Python to follow along; the Python primer in Chapter 1 introduces the constructs that recur throughout. If you only have an hour, read Chapter 1, Chapter 2 (`_shared.py`), and Chapter 6 (Firth + conformal). Those three pieces are the spine of the analysis.

Contents

1	A short Python primer for clinicians	2
1.1	How a Python file is organised	2
1.2	Variables, lists, and dictionaries	2
1.3	Functions: code with a name and inputs	2
1.4	Pandas, numpy, and scikit-learn	3
1.5	Pipelines and the scikit-learn fit / predict pattern	3
1.6	Cross-validation in three lines	3
2	_shared.py — the foundations every script depends on	4
2.1	Imports and global thread caps	4
2.2	Path constants	4
2.3	Feature lists	4
2.4	The loader functions	5
2.5	Pipeline factories	6
2.6	Out-of-fold prediction helper	6
3	02_calibration.py — measuring how trustworthy the probabilities are	8
4	04_loho.py — leave-one-hospital-out validation	10
5	14_decision_tree.py — the TreeAge-style cost-effectiveness model	13

6	24_firth_bayes_lr.py — the deployment model	16
7	25_conformal_prediction.py — turning probabilities into bedside decisions	20
8	16_voi_evpi.py — value-of-information analysis	23
9	29_main_figures_jnnp.py — publication-grade figures	26
10	27_build_jnnp_manuscript.py — building the Word manuscript from code	29
11	30_export_calculator_assets.py — packaging the model for the dashboard	31
12	The interactive dashboards — HTML, CSS, JavaScript	34
12.1	Anatomy of an HTML page	34
12.2	How the risk calculator works	34
12.3	How the savings calculator works	35
12.4	How the interactive callgraph works	36
13	The callgraph — how the scripts depend on each other	37
14	Other scripts — what they do, in one paragraph each	38
14.1	03_dca.py	38
14.2	05_temporal_leakage.py	38
14.3	06_overfitting.py	38
14.4	07_missing_data.py	38
14.5	08_eicu_cohort.py	38
14.6	09_competing_risks.py	38
14.7	10_11_cea_pairwise.py	38
14.8	12_nis_seizure_reclassify.py	38
14.9	13_nis_grouped_lasso.py	38
14.10	15_radiology_nlp.py	39
14.11	17_build_slides.py	39
14.12	18_bidmc_optimize.py	39
14.13	19_transfer_learning.py	39
14.14	20_build_manuscript.py	39
14.15	21_imbalance_sweep.py	39
14.16	22_diverse_stacking.py	39
14.17	23_tabpfn_eval.py	39
14.18	26_main_figures.py	39
14.19	28_make_callgraph.py	39
14.20	31_export_callgraph_json.py	39
14.21	32_build_code_companion.py	39
14.22	33_build_code_companion_pdf.py	40
15	Further study — Claude skills that help you learn the code	41

1 A short Python primer for clinicians

Python is a programming language often described as 'executable pseudocode' because the constructs in a Python script tend to translate cleanly into English sentences. This chapter introduces the small set of building blocks that recur throughout our analysis code.

1.1 How a Python file is organised

A Python file (extension .py) is read from top to bottom. The topmost few lines state the purpose of the file in a triple-quoted docstring, then import the libraries the file will need.

```
1 """Task 24 – Firth penalized + Bayesian logistic regression.
2 A docstring describes what this file does and what it produces.
3 """
4 import os, sys
5 import numpy as np, pandas as pd
6 from sklearn.linear_model import LogisticRegression
7 from _shared import load_bidmc, RES, FIG, SEED
```

Lines starting with the word import bring in code from elsewhere. import numpy as np gives us a shorthand: from this line on, np stands for the numpy library. from sklearn.linear_model import LogisticRegression pulls one specific tool out of a larger library — exactly like importing one drug class from a pharmacopoeia.

1.2 Variables, lists, and dictionaries

```
1 # A single value
2 age = 73
3 prevalence = 0.073
4 cohort_name = "BIDMC"
5
6 # A list – an ordered collection
7 features = ["age", "sex", "preop_gcs", "sdh_thickness"]
8 features.append("mid_shift")
9
10 # A dictionary – a mapping from keys to values
11 feature_means = {"age": 73, "sex": 0.68, "preop_gcs": 14.1}
12 feature_means["epilepsy_hx"] = 0.02
```

Variables are labels that point to values. Lists hold ordered collections (square brackets). Dictionaries map a key to a value (curly braces). The hash sign # introduces a comment.

1.3 Functions: code with a name and inputs

```
1 def bootstrap_auc(y, p, n_boot=1000, seed=42):
2     """Compute the AUC and a 95% bootstrap confidence interval."""
3     rng = np.random.default_rng(seed)
4     bs = []
5     for _ in range(n_boot):
6         idx = rng.integers(0, len(y), len(y))
```

```

7     bs.append(roc_auc_score(y[idx], p[idx]))
8     lo, hi = np.percentile(bs, [2.5, 97.5])
9     return float(roc_auc_score(y, p)), float(lo), float(hi)

```

A function is a named recipe. `def` starts the definition; the name follows; the inputs go inside the parentheses. Inputs can have default values. The indented block under `def` is the body. The `for` line is a loop. The `return` statement at the end is what the function gives back to its caller.

1.4 Pandas, numpy, and scikit-learn

Three external libraries do almost all of the analytical work in this codebase. `numpy` provides fast numerical arrays. `pandas` provides DataFrames (tables with named columns and labelled rows). `scikit-learn` (imported as `sklearn`) provides the machine-learning models, cross-validation splitters, and the standardisation machinery.

1.5 Pipelines and the scikit-learn fit / predict pattern

```

1 pipe = Pipeline([
2     ("imputer", SimpleImputer(strategy="median")),
3     ("scaler", StandardScaler()),
4     ("classifier", FirthLogisticRegression()),
5 ])
6 pipe.fit(X_train, y_train)
7 probs = pipe.predict_proba(X_test)[: , 1]

```

A scikit-learn pipeline is a chain of steps. When we call `fit`, the pipeline runs imputation on the training data, then standardisation, then trains the classifier. When we call `predict_proba`, the pipeline runs the same imputation and standardisation (using the training-time parameters) and then asks the classifier for predicted probabilities.

A pipeline guarantees that the same preprocessing applied to the training data is applied — using the training-time statistics — to any new data. Without a pipeline it is easy to compute the column means on the test set and introduce a leakage bias.

1.6 Cross-validation in three lines

```

1 rskf = RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=42)
2 for train_idx, test_idx in rskf.split(X, y):
3     pipe.fit(X.iloc[train_idx], y.iloc[train_idx])
4     p_oof[test_idx] = pipe.predict_proba(X.iloc[test_idx])[:, 1]

```

`RepeatedStratifiedKFold` yields pairs of (`train_idx`, `test_idx`). Each pair is one fold: train on 80% of the data, evaluate on the held-out 20%, save the predicted probabilities for the held-out patients into `p_oof` (out-of-fold). After all folds are processed, `p_oof` contains one prediction for every patient.

2 `_shared.py` — the foundations every script depends on

`_shared.py` is the file every other script in the project imports. It defines path constants, feature lists, loader functions, and pipeline factories. Reading this file is the single highest-yield investment of your time, because the patterns it establishes appear in every other script.

2.1 Imports and global thread caps

The file opens with environment-variable settings that force the underlying numerical libraries (OpenBLAS, MKL, OpenMP) to use a single CPU thread per process. On Apple Silicon, multi-threaded scikit-learn can deadlock; setting `n_jobs=1` throughout the codebase prevents that.

```
1  """Shared utilities for revision analyses.
2
3  n_jobs is forced to 1 throughout for Apple Silicon stability.
4  """
5  import os, sys, warnings, json
6  from pathlib import Path
7  import numpy as np
8  import pandas as pd
9  import joblib
10
11  warnings.filterwarnings("ignore")
12
13  ROOT = Path(__file__).resolve().parents[2]
14  OUT = ROOT / "revision_analyses"
15  RES = OUT / "results"
16  FIG = OUT / "figures"
17  CACHE = OUT / "cache"
18  for p in (RES, FIG, CACHE):
19      p.mkdir(parents=True, exist_ok=True)
20
21  SEED = 42
22  N_JOBS = 1 # rationale: Apple-Silicon thread-safety on scikit-learn 1.5.x;
23             # multi-threaded fits deadlock under macOS Accelerate. Never raise.
24  ... (381 more lines elided – see full script in the repository) ...
```

2.2 Path constants

ROOT is the project's top-level folder. OUT is `revision_analyses/`, RES is its `results/` subfolder, FIG is `figures/`, CACHE is for intermediate arrays. The for loop creates each folder if missing.

2.3 Feature lists

POSTOP_A_FEATURES is a list of 21 column names. Defining it here, once, means every script that fits `postop_A` uses exactly the same 21 predictors. POSTOP_B_FEATURES is the same list with the three leakage-suspect variables removed — the temporal-leakage robustness analysis.

```
1  POSTOP_A_FEATURES = [
2      "sex", "age", "blood_type", "sdh_type", "sdh_thickness", "csdh_size_change",
```

```

3     "mid_shift", "hematoma_lat", "collection_density", "preop_gcs",
4     "epilepsy_hx", "num_prev_sdh", "demographic", "procedures",
5     "surg_decompression", "mma_embo", "drainage", "postop_gcs",
6     "aed_timing_recoded", "prop_aed", "ab_eeg",
7 ]
8 POSTOP_B_FEATURES = [
9     "sex", "age", "blood_type", "sdh_type", "sdh_thickness", "csdh_size_change",
10    "mid_shift", "hematoma_lat", "collection_density", "preop_gcs",
11    "epilepsy_hx", "num_prev_sdh", "demographic", "procedures",
12    "surg_decompression", "mma_embo", "drainage", "postop_gcs",
13 ]
14 PREOP_FEATURES = [
15     "sex", "age", "blood_type", "sdh_type", "sdh_thickness", "csdh_size_change",
16     "mid_shift", "hematoma_lat", "collection_density", "preop_gcs",
17     "epilepsy_hx", "num_prev_sdh", "demographic", "procedures",
18 ]
19
20 def load_bidmc():
21     df = pd.read_csv(BIDMC_CSV)
22     df["aed_timing_recoded"] = df["aed_timing"].replace({2: 1})
23     return df
24
25 ... (345 more lines elided – see full script in the repository) ...

```

2.4 The loader functions

load_bidmc reads the cleaned BIDMC CSV and applies a single recoding. load_eicu does the same for eICU. load_eicu_pure adds an extra filter for the pure post-craniotomy sensitivity cohort.

```

1 def load_bidmc():
2     df = pd.read_csv(BIDMC_CSV)
3     df["aed_timing_recoded"] = df["aed_timing"].replace({2: 1})
4     return df
5
6 # -----
7 # eICU
8 # -----
9 EICU_CSV = ROOT / "eicu_csdh_cohort_final.csv"
10
11 EICU_SET_A = ["age", "sex", "gcs_admission", "prior_seizures", "craniotomy",
12              "burr_hole", "prophylactic_aed", "pre_admission_aed"]
13
14 EICU_SET_B_ADD = ["apache_score", "apache_gcs", "any_anticoagulant", "any_antiplaetlet",
15                  "any_steroid", "mechanical_ventilation", "icp_monitor", "blood_transfusion",
16                  "comorbidity_count", "hx_hypertension", "hx_diabetes", "hx_stroke",
17                  "hx_dementia", "hx_afib", "hx_alcohol", "pupil_abnormality",
18                  "icp_available", "n_labs_24h", "n_vitals_24h", "Hgb_first",
19                  "PT__INR_first", "WBC_x_1000_first", "albumin_first", "calcium_first",
20                  "creatinine_first", "glucose_first", "lactate_first", "magnesium_first",
21                  "platelets_x_1000_first", "potassium_first", "sodium_first",
22                  "total_bilirubin_first", "temperature_mean", "sao2_mean",
23                  "heartrate_mean", "respiration_mean", "systemicsystolic_mean",
24                  "systemicdiastolic_mean", "systemicmean_mean"]

```

```

25
26 EICU_SET_C_ADD = ["gcs_24h", "gcs_min_24h", "gcs_max_24h", "gcs_delta", "n_gcs_assessments",
27                  "sodium_slope_48h", "sodium_delta_48h", "sodium_cv_48h",
28                  "potassium_slope_48h", "potassium_delta_48h", "potassium_cv_48h",
29                  "glucose_slope_48h", "glucose_delta_48h", "glucose_cv_48h",
30                  "creatinine_slope_48h", "creatinine_delta_48h", "creatinine_cv_48h",
31                  "platelets_x_1000_slope_48h", "platelets_x_1000_delta_48h",
32                  "platelets_x_1000_cv_48h",
33                  "lactate_slope_48h", "lactate_delta_48h", "lactate_cv_48h",
34                  "WBC_x_1000_slope_48h", "WBC_x_1000_delta_48h", "WBC_x_1000_cv_48h",
35                  "Hgb_slope_48h", "Hgb_delta_48h", "Hgb_cv_48h",
36                  "heartrate_slope_48h", "heartrate_delta_48h", "heartrate_cv_48h",
37                  "systemicsystolic_slope_48h", "systemicsystolic_delta_48h",
38                  "systemicsystolic_cv_48h",
39                  "systemicmean_slope_48h", "systemicmean_delta_48h", "systemicmean_cv_48h",
40                  "temperature_slope_48h", "temperature_delta_48h", "temperature_cv_48h",
41                  "sao2_slope_48h", "sao2_delta_48h", "sao2_cv_48h",
42                  "temperature_min", "temperature_max", "temperature_std",
43                  ... (310 more lines elided – see full script in the repository) ...

```

2.5 Pipeline factories

Each pipeline factory returns a fresh, untrained scikit-learn Pipeline. Returning a fresh pipeline matters because pipelines are mutable; fitting one in a loop would carry over state from the previous iteration.

```

1 def make_pipeline_postopA():
2     from sklearn.pipeline import Pipeline
3     from sklearn.compose import ColumnTransformer
4     from sklearn.impute import SimpleImputer
5     from sklearn.preprocessing import StandardScaler
6     from imblearn.ensemble import BalancedRandomForestClassifier
7     prep = ColumnTransformer(
8         [("num", Pipeline([("imp", SimpleImputer(strategy="median")),
9                           ("sc", StandardScaler())])),
10        ])
11     clf = BalancedRandomForestClassifier(
12         # rationale: n_estimators=300 matches the prior version of this manuscript; tuning
13         # showed diminishing returns past ~300 trees on n=655.
14         # min_samples_leaf=2 is the imbalanced-learn default for BRF and
15         # leaves enough leaves to keep variable-importance ranks stable
16         # (Spearman  $\rho = 0.98$  across folds in 06_overfitting.py).
17         n_estimators=300, min_samples_leaf=2,
18         n_jobs=N_JOBS, random_state=SEED,
19     )
20     return Pipeline([("prep", prep), ("clf", clf)])

```

2.6 Out-of-fold prediction helper

```

1 def oof_predictions(make_pipe_fn, X, y, n_splits=5, n_repeats=5, groups=None):
2     """Return mean out-of-fold probability for each row across repeated
3     stratified K-fold cross-validation.

```

```

4
5     Parameters
6     -----
7     make_pipe_fn : Callable returning a fresh sklearn Pipeline.
8     X : pd.DataFrame - feature matrix.
9     y : pd.Series    - binary outcome.
10    n_splits : int    - number of folds per repeat (default 5).
11    n_repeats : int   - number of repeated draws of the K-fold split (default 5).
12    groups : unused, kept for backward-compatible signature.
13
14    Returns
15    -----
16    np.ndarray of length len(X) with the average OOF probability per row.
17    ... (12 more lines elided - see full script in the repository) ...

```

This function returns one predicted probability per patient by running repeated stratified K-fold cross-validation and averaging the predictions whenever a patient appears in multiple test folds. The accumulator trick — adding into `p_acc` and incrementing `n_acc` — is the standard idiom for averaging without storing every individual prediction.

3 02_calibration.py — measuring how trustworthy the probabilities are

AUC measures rank-order discrimination — does the model assign higher probabilities to patients who go on to seize? Calibration measures whether those probabilities are themselves trustworthy — when the model says 10%, do roughly 10% of those patients actually seize? Calibration is the metric that shapes clinical decisions.

```
1  """Task 2 – Calibration analysis (BIDMC + eICU).
2
3  Outputs:
4      results/02_calibration_metrics.csv
5      figures/02_calibration_curves.png
6      cache/oof_<model>.npz    (reused by DCA / temporal-leakage / etc.)
7  """
8  import sys, os
9  sys.path.insert(0, os.path.dirname(__file__))
10 os.environ["OMP_NUM_THREADS"] = "1"
11 os.environ["OPENBLAS_NUM_THREADS"] = "1"
12 os.environ["MKL_NUM_THREADS"] = "1"
13
14 import numpy as np, pandas as pd, json
15 import matplotlib; matplotlib.use("Agg")
16 import matplotlib.pyplot as plt
17 from _shared import (
18     load_bidmc, load_eicu, load_eicu_pure,
19     POSTOP_A_FEATURES, POSTOP_B_FEATURES, PREOP_FEATURES,
20     EICU_SET_A, EICU_SET_B, EICU_SET_C,
21     make_pipeline_postopA, make_pipeline_postopB, make_pipeline_preop,
22     make_pipeline_eicu, oof_predictions, calibration_metrics,
23     RES, FIG, CACHE, SEED,
24 )
25
26 def bootstrap_calibration_ci(y, p, n_boot=1000, seed=42):
27     """Fix F: paired-bootstrap 95% CI for Brier, CITL, slope/intercept, AUC."""
28     from sklearn.metrics import brier_score_loss, roc_auc_score
29     from sklearn.linear_model import LogisticRegression
30     rng = np.random.default_rng(seed)
31     n = len(y); collect = {k: [] for k in ("brier", "citl", "slope", "intercept", "auc")}
32     for _ in range(n_boot):
33         idx = rng.integers(0, n, n)
34         yb, pb = y[idx], p[idx]
35         if len(np.unique(yb)) < 2: continue
36         # Brier
37         collect["brier"].append(brier_score_loss(yb, pb))
38         # CITL = observed - mean predicted
39         collect["citl"].append(yb.mean() - pb.mean())
40         # AUC
41         collect["auc"].append(roc_auc_score(yb, pb))
42         # slope / intercept via logit-regression of outcome on logit(pred)
43         p_safe = np.clip(pb, 1e-6, 1 - 1e-6)
44         x_logit = np.log(p_safe / (1 - p_safe)).reshape(-1, 1)
45         try:
```

```

46         lr = LogisticRegression(C=1e6, solver="lbfgs", max_iter=2000).fit(x_logit, yb)
47         collect["intercept"].append(float(lr.intercept_[0]))
48         collect["slope"].append(float(lr.coef_[0, 0]))
49     except Exception:
50         pass
51     ci = {}
52     for k, v in collect.items():
53         if v:
54             lo, hi = np.percentile(v, [2.5, 97.5])
55             ci[f"{k}_lo"] = float(lo); ci[f"{k}_hi"] = float(hi)
56         else:
57             ci[f"{k}_lo"] = float("nan"); ci[f"{k}_hi"] = float("nan")
58     return ci
59
60 def run_one(name, make_pipe, X, y, n_splits=5, n_repeats=5):
61     print(f" {name}: n={len(y)}, events={int(y.sum())}, splits={n_splits}x{n_repeats}",
62           flush=True)
63     p = oof_predictions(make_pipe, X, y, n_splits=n_splits, n_repeats=n_repeats)
64     np.savez(CACHE / f"oof_{name}.npz", y=y.values, p=p)
65     m = calibration_metrics(y.values, p)
66     m["model"] = name
67     m["n"] = int(len(y))
68     m["events"] = int(y.sum())
69     # Fix F: bootstrap 95% CIs on the core calibration metrics
70     m.update(bootstrap_calibration_ci(y.values, p, n_boot=1000, seed=42))
71     return m, p
72
73 def main():
74     rows = []
75     cohorts = {}
76
77     # BIDMC
78     df = load_bidmc()
79     y = df["seizure"].astype(int)
80     print(f"\nBIDMC n={len(df)}, events={y.sum()}")
81     m, p = run_one("bidmc_postopA",
82 ... (90 more lines elided – see full script in the repository) ...

```

The metrics are computed by `calibration_metrics` in `_shared.py`: Brier score, calibration-in-the-large, calibration slope, intercept, ECE, MCE, and the Hosmer-Lemeshow chi-square. A well-calibrated model has Brier near $p \times (1-p)$, CITL near zero, slope near one, and intercept near zero.

4 04_loho.py — leave-one-hospital-out validation

External validation in eICU is more demanding than ordinary cross-validation because it asks: does the model generalise from hospital A to hospital B? We answer that by leaving one hospital out at a time, training on the remaining 138, and scoring the held-out hospital.

```
1  """Task 4 – Leave-one-hospital-out CV on eICU.
2
3  For each hospital with >= MIN_EVENTS seizures, train on remaining 138 hospitals
4  and evaluate on the held-out hospital. Report AUC distribution.
5
6  Outputs:
7      results/04_loho_per_hospital.csv
8      results/04_loho_summary.csv
9      figures/04_loho_distribution.png
10 """
11 import sys, os
12 sys.path.insert(0, os.path.dirname(__file__))
13 os.environ["OMP_NUM_THREADS"] = "1"
14 os.environ["OPENBLAS_NUM_THREADS"] = "1"
15 os.environ["MKL_NUM_THREADS"] = "1"
16
17 import numpy as np, pandas as pd
18 import matplotlib; matplotlib.use("Agg")
19 import matplotlib.pyplot as plt
20 from sklearn.metrics import roc_auc_score, average_precision_score, brier_score_loss
21 from _shared import (
22     load_eicu, load_eicu_pure, EICU_SET_A, EICU_SET_C,
23     make_pipeline_eicu, RES, FIG, CACHE, SEED,
24 )
25
26 MIN_EVENTS = 3  # held-out hospital must have ≥3 seizures for AUC
27 MIN_PATIENTS = 10
28
29 def make_pipe_light(features):
30     """Lighter pipeline (200 trees) for LOHO speed."""
31     from sklearn.pipeline import Pipeline
32     from sklearn.compose import ColumnTransformer
33     from sklearn.impute import SimpleImputer
34     from sklearn.preprocessing import StandardScaler
35     from sklearn.ensemble import RandomForestClassifier
36     prep = ColumnTransformer(
37         [("num", Pipeline([("imp", SimpleImputer(strategy="median")),
38                             ("sc", StandardScaler())]), features)]
39     )
40     clf = RandomForestClassifier(
41         n_estimators=200, min_samples_leaf=2, class_weight="balanced",
42         n_jobs=1, random_state=SEED,
43     )
44     return Pipeline([("prep", prep), ("clf", clf)])
45
46 def loho_for(df, features, set_name, cohort_name, ckpt_path):
47     """Returns a DataFrame; supports incremental checkpointing."""
```

```

48 y = df["seizure"].astype(int).values
49 H = df["hospitalid"].values
50 X = df[features]
51 hospitals = sorted(df["hospitalid"].unique())
52 print(f" {cohort_name} / {set_name}: {len(hospitals)} hospitals, n={len(df)},
    sz={y.sum()}", flush=True)
53
54 if ckpt_path.exists():
55     rows = pd.read_csv(ckpt_path).to_dict("records")
56     done = {r["hospitalid"] for r in rows
57             if r["cohort"] == cohort_name and r["set"] == set_name}
58     print(f" resuming, {len(done)} hospitals done", flush=True)
59 else:
60     rows = []; done = set()
61
62 for i, h in enumerate(hospitals):
63     if int(h) in done:
64         continue
65     te = (H == h)
66     tr = ~te
67     if y[te].sum() < MIN_EVENTS or te.sum() < MIN_PATIENTS:
68         continue
69     if y[tr].sum() == 0 or (1 - y[tr]).sum() == 0:
70         continue
71     pipe = make_pipe_light(features)
72     pipe.fit(X[tr], y[tr])
73     p = pipe.predict_proba(X[te])[:, 1]
74     auc = roc_auc_score(y[te], p)
75     prauc = average_precision_score(y[te], p)
76     brier = brier_score_loss(y[te], p)
77     rows.append({
78         "cohort": cohort_name, "set": set_name, "hospitalid": int(h),
79         "n_test": int(te.sum()), "events_test": int(y[te].sum()),
80         "auc": auc, "prauc": prauc, "brier": brier,
81     })
82     # incremental checkpoint
83     pd.DataFrame(rows).to_csv(ckpt_path, index=False)
84     if len(rows) % 5 == 0:
85         print(f" {len(rows)} hospitals scored, last AUC={auc:.3f}", flush=True)
86 return pd.DataFrame(rows)
87
88 def main():
89     df_full = load_eicu()
90     df_pure = load_eicu_pure()
91     print(f"Full: n={len(df_full)} events={df_full.seizure.sum()}
    hospitals={df_full.hospitalid.nunique()}")
92     print(f"Pure: n={len(df_pure)} events={df_pure.seizure.sum()}
    hospitals={df_pure.hospitalid.nunique()}")
93
94     ckpt = CACHE / "04_loho_ckpt.csv"
95     parts = []
96     parts.append(loho_for(df_full, EICU_SET_A, "Set_A", "full", ckpt))
97     parts.append(loho_for(df_full, EICU_SET_C, "Set_C", "full", ckpt))
98     parts.append(loho_for(df_pure, EICU_SET_C, "Set_C", "pure", ckpt))

```

```

99     perh = pd.concat(parts, ignore_index=True).drop_duplicates(["cohort","set","hospitalid"])
100     perh.to_csv(RES / "04_loho_per_hospital.csv", index=False)
101
102     # — Fix D: Random-effects meta-analytic pooling (DerSimonianLaird)
103     # Treat each held-out hospital AUC as one study; transform to logit-AUC,
104     # compute within-study variance via HanleyMcNeil, then DL estimator for
105     # between-hospital heterogeneity  $\tau^2$ .
106     def hanley_mcneil_var(auc, n1, n0):
107         """Approximate variance of AUC (Hanley & McNeil 1982)."""
108         auc = float(np.clip(auc, 1e-6, 1 - 1e-6))
109         n1 = max(int(n1), 1); n0 = max(int(n0), 1)
110         Q1 = auc / (2.0 - auc); Q2 = 2.0 * auc**2 / (1.0 + auc)
111         return (auc * (1 - auc)
112                 + (n1 - 1) * (Q1 - auc**2)
113                 + (n0 - 1) * (Q2 - auc**2)) / (n1 * n0)
114
115     def logit(p): return np.log(p / (1 - p))
116     def invlogit(x): return 1.0 / (1.0 + np.exp(-x))
117
118     def random_effects_pool(g):
119         """DerSimonianLaird pooling on logit-AUC scale."""
120         auc = g["auc"].values
121     ... (104 more lines elided – see full script in the repository) ...

```

loho_for handles the leave-one-hospital-out loop and writes a CSV row per hospital. random_effects_pool turns those per-hospital AUCs into a single meta-analytic estimate using the DerSimonian-Laird method.

5 14_decision_tree.py — the TreeAge-style cost-effectiveness model

Decision-tree models in health economics compute the expected cost and expected QALYs of a strategy by enumerating every possible patient trajectory, weighting each by its probability, and adding the results. This script implements that for our four strategies and renders Figure 4.

```
1  """Task 14 – TreeAge-style decision tree (Figure 4 for CEA).
2
3  Renders the 4-strategy decision tree with:
4      □ decision node (root)
5      ○ chance nodes (seizure y/n, detected y/n)
6      ▷ terminal nodes (cost / QALY pair)
7
8  Uses the deterministic point-estimate parameters from 10_11_cea_pairwise.py and
9  performs expected-value rollback at each chance node so the tree shows both the
10 structure and the rolled-up E[Cost], E[QALY] per strategy.
11
12 Outputs:
13     figures/14_decision_tree.{png,pdf,svg}
14     results/14_decision_tree_rollback.csv
15 """
16 import sys, os
17 sys.path.insert(0, os.path.dirname(__file__))
18 os.environ["OMP_NUM_THREADS"] = "1"
19
20 import matplotlib; matplotlib.use("Agg")
21 import matplotlib.pyplot as plt
22 import matplotlib.patches as mpatches
23 from matplotlib.patches import FancyBboxPatch
24 import pandas as pd
25
26 from _shared import RES, FIG
27
28 # — Base-case parameters (point estimates from Params in 10_11_cea_pairwise.py)
29 # Kept here as plain dict for readability in the figure.
30 P = dict(
31     p_seizure_base    = 0.12,
32     aed_rrr           = 0.45,
33     model_sens         = 0.842,
34     model_spec         = 0.504,
35     p_detect_ceeg      = 0.95,
36     p_detect_clin      = 0.30,
37     cost_ml            = 50.0,
38     cost_aed_total     = 200.0 + 1507.0 + 441.0,
39     cost_aed_total_ceeg = 200.0 + 1293.0 + 140.0,
40     cost_ceeg_total    = 1500.0 * 3,
41     cost_sz_detected   = 2500.0 + (20.3 - 9.8) * 3500.0 + 1000.0,
42     cost_sz_undet_mult = 1.3,
43     qaly_baseline      = 0.75,
44     qaly_sz_loss       = 0.10,
45     qaly_aed_loss      = 0.02 * (66 / 365),
46     qaly_aed_loss_ceeg = 0.02 * (21 / 365),
47     horizon_qaly_no_sz = 7.5,    # ~10y * 0.75 discounted
```

```

48     horizon_qaly_sz      = 6.8,
49     horizon_qaly_undet = 6.3,
50 )
51
52 # — Strategy logic – point-estimate rollback —————
53 def rollback_strategy(name):
54     """Return list of leaves: (path_label, prob, cost, qaly)."""
55     p_sz = P["p_seizure_base"]
56     leaves = []
57
58     if name == "obs":
59         # No prophylaxis, clinical detection only
60         p_det = P["p_detect_clin"]
61         leaves.append((f"→szdet", p_sz * p_det,
62                       P["cost_sz_detected"],
63                       P["horizon_qaly_sz"] - P["qaly_sz_loss"]))
64         leaves.append((f"→szmiss", p_sz * (1 - p_det),
65                       P["cost_sz_detected"] * P["cost_sz_undet_mult"],
66                       P["horizon_qaly_undet"] - P["qaly_sz_loss"]))
67         leaves.append((f"no sz", 1 - p_sz, 0.0,
68                       P["horizon_qaly_no_sz"]))
69
70     elif name == "aed_all":
71         # Universal AED, clinical detection
72         p_sz_treated = p_sz * (1 - P["aed_rrr"])
73         p_det = P["p_detect_clin"]
74         leaves.append((f"→AEDszdet", p_sz_treated * p_det,
75                       P["cost_aed_total"] + P["cost_sz_detected"],
76                       P["horizon_qaly_sz"] - P["qaly_sz_loss"] - P["qaly_aed_loss"]))
77         leaves.append((f"→AEDszmiss", p_sz_treated * (1 - p_det),
78                       P["cost_aed_total"] + P["cost_sz_detected"] * P["cost_sz_undet_mult"],
79                       P["horizon_qaly_undet"] - P["qaly_sz_loss"] - P["qaly_aed_loss"]))
80         leaves.append((f"→AEDno sz", 1 - p_sz_treated,
81                       P["cost_aed_total"],
82                       P["horizon_qaly_no_sz"] - P["qaly_aed_loss"]))
83
84     elif name == "ml_aed":
85         # ML → AED if predicted high risk; clinical detection
86         sens = P["model_sens"]; spec = P["model_spec"]
87         tp = sens * p_sz; fn = (1 - sens) * p_sz
88         fp = (1 - spec) * (1 - p_sz); tn = spec * (1 - p_sz)
89         rrr = P["aed_rrr"]
90         # TP: AED reduces seizure risk by rrr; if seizes, detected clinically
91         leaves.append(("ML→TPprevent", tp * rrr,
92                       P["cost_ml"] + P["cost_aed_total"],
93                       P["horizon_qaly_no_sz"] - P["qaly_aed_loss"]))
94         leaves.append(("ML→TPsz", tp * (1 - rrr),
95                       P["cost_ml"] + P["cost_aed_total"] + P["cost_sz_detected"],
96                       P["horizon_qaly_sz"] - P["qaly_sz_loss"] - P["qaly_aed_loss"]))
97         # FP: AED but never would have seized
98         leaves.append(("ML→FP", fp,
99                       P["cost_ml"] + P["cost_aed_total"],
100                      P["horizon_qaly_no_sz"] - P["qaly_aed_loss"]))
101         # FN: no AED, seizure occurs, clinical detection

```

```

102     p_det = P["p_detect_clin"]
103     leaves.append(("→MLFNdet", fn * p_det,
104                   P["cost_ml"] + P["cost_sz_detected"],
105                   P["horizon_qaly_sz"] - P["qaly_sz_loss"]))
106     leaves.append(("→MLFNmiss", fn * (1 - p_det),
107                   P["cost_ml"] + P["cost_sz_detected"] * P["cost_sz_undet_mult"],
108                   P["horizon_qaly_undet"] - P["qaly_sz_loss"]))
109     # TN: no AED, no seizure
110     leaves.append(("→MLTN", tn,
111                   P["cost_ml"],
112                   P["horizon_qaly_no_sz"]))
113
114     elif name == "ml_ceeg":
115         # ML → cEEG monitoring if predicted high risk
116         sens = P["model_sens"]; spec = P["model_spec"]
117         tp = sens * p_sz; fn = (1 - sens) * p_sz
118         fp = (1 - spec) * (1 - p_sz); tn = spec * (1 - p_sz)
119         p_det = P["p_detect_ceeg"] # cEEG much higher detection
120         # High-risk branch: cEEG + targeted AED (lower-burden)
121     ... (188 more lines elided – see full script in the repository) ...

```

rollback_strategy returns the list of terminal leaves. compute_ev does the rollback: probability times payoff, summed. render_tree draws the tree in matplotlib using the TreeAge visual convention.

6 24_firth_bayes_lr.py — the deployment model

With 48 events in BIDMC, ordinary maximum-likelihood logistic regression is unstable. Firth's penalty stabilises the coefficient estimates and produces valid confidence intervals even with rare events.

```
1  """Task 24 – Firth penalized LR + Bayesian LR with priors derived from eICU.
2
3  For rare-event clinical prediction (48/655), maximum-likelihood logistic regression
4  suffers from separation bias and inflated coefficient SEs. Two principled fixes:
5
6  (a) Firth (1993) penalized likelihood logistic regression
7      – Adds Jeffreys-prior penalty:  $\hat{\beta} = \hat{\beta} + 0.5 \log|I\hat{\beta}|$ 
8      – Demonstrated to stabilize estimation when events_per_variable < 10
9      – Puhr et al., Stat Med 2017 (arXiv:2101.07620)
10
11  (b) Bayesian logistic regression with informative priors derived from the
12      n=5,376 eICU cohort
13      – Step 1: fit elastic-net LR on eICU using overlapping shared features
14      – Step 2: use eICU coefficient point estimates as Gaussian prior means
15        on the corresponding BIDMC coefficients
16      – Step 3: posterior mode by penalized IRLS with diagonal Gaussian prior
17        precision (acts as L2 with non-zero center)
18      – This is a principled alternative to "transfer learning" that does not
19        require shared feature names beyond the prior-informed subset.
20
21  Evaluation: 5x5 repeated stratified CV; bootstrap 95% AUC CI; paired DeLong
22  test vs the BalancedRandomForest baseline.
23
24  Outputs:
25      results/24_firth_bayes_lr.csv
26      figures/24_firth_bayes_lr.{png,pdf}
27  """
28  import sys, os
29  sys.path.insert(0, os.path.dirname(__file__))
30  os.environ["OMP_NUM_THREADS"] = "1"
31
32  import warnings; warnings.filterwarnings("ignore")
33  import numpy as np, pandas as pd
34  import matplotlib; matplotlib.use("Agg")
35  import matplotlib.pyplot as plt
36
37  from sklearn.model_selection import RepeatedStratifiedKFold
38  from sklearn.metrics import roc_auc_score, brier_score_loss, average_precision_score
39  from sklearn.compose import ColumnTransformer
40  from sklearn.pipeline import Pipeline
41  from sklearn.impute import SimpleImputer
42  from sklearn.preprocessing import StandardScaler
43  from sklearn.linear_model import LogisticRegression
44  from imblearn.ensemble import BalancedRandomForestClassifier
45
46  from firthlogist import FirthLogisticRegression
47
48  from _shared import (load_bidmc, load_eicu, POSTOP_A_FEATURES, POSTOP_B_FEATURES,
49                      RES, FIG, CACHE, SEED)
```

```

50
51 N_SPLITS, N_REPEATS = 5, 5
52
53 # Shared features for eICU prior derivation (match BIDMC names → eICU names)
54 SHARED_BIDMC_NAMES = ["age", "sex", "preop_gcs", "epilepsy_hx", "prop_aed"]
55 SHARED_EICU_NAMES = ["age", "sex", "gcs_admission", "prior_seizures", "prophylactic_aed"]
56
57 def make_prep(features):
58     return ColumnTransformer([
59         ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
60             ("sc", StandardScaler())])), features)])
61
62 def cv_oof(make_pipe_fn, X, y, n_splits=N_SPLITS, n_repeats=N_REPEATS):
63     rskf = RepeatedStratifiedKFold(n_splits=n_splits, n_repeats=n_repeats, random_state=SEED)
64     p_acc = np.zeros(len(X)); n_acc = np.zeros(len(X))
65     for tr, te in rskf.split(X, y):
66         try:
67             p = make_pipe_fn()
68             p.fit(X.iloc[tr], y.iloc[tr])
69             prob = p.predict_proba(X.iloc[te])[:, 1]
70             p_acc[te] += prob; n_acc[te] += 1
71         except Exception:
72             continue
73     return p_acc / np.maximum(n_acc, 1)
74
75 def bootstrap_auc(y, p, n_boot=1000, seed=SEED):
76     rng = np.random.default_rng(seed); bs = []
77     n = len(y)
78     for _ in range(n_boot):
79         idx = rng.integers(0, n, n)
80         if len(np.unique(y[idx])) < 2: continue
81         bs.append(roc_auc_score(y[idx], p[idx]))
82     lo, hi = np.percentile(bs, [2.5, 97.5]) if bs else (np.nan, np.nan)
83     return float(roc_auc_score(y, p)), float(lo), float(hi)
84
85 def delong_test(y, p1, p2):
86     y = np.asarray(y); p1 = np.asarray(p1); p2 = np.asarray(p2)
87     pos = (y == 1); neg = (y == 0)
88     m, n = pos.sum(), neg.sum()
89     if m < 2 or n < 2: return float("nan"), float("nan")
90     def struct(s):
91         sp = s[pos]; sn = s[neg]
92         V10 = np.array([(np.sum(sn < v) + 0.5 * np.sum(sn == v)) / n for v in sp])
93         V01 = np.array([(np.sum(sp > v) + 0.5 * np.sum(sp == v)) / m for v in sn])
94         return V10.mean(), V10, V01
95     a1, v10_1, v01_1 = struct(p1); a2, v10_2, v01_2 = struct(p2)
96     s10 = (np.var(v10_1, ddof=1) + np.var(v10_2, ddof=1)
97         - 2 * np.cov(v10_1, v10_2, ddof=1)[0,1])
98     s01 = (np.var(v01_1, ddof=1) + np.var(v01_2, ddof=1)
99         - 2 * np.cov(v01_1, v01_2, ddof=1)[0,1])
100     var_diff = s10 / m + s01 / n
101     if var_diff <= 0: return float("nan"), float("nan")
102     z = (a1 - a2) / np.sqrt(var_diff)
103     from scipy.stats import norm

```

```

104     return float(z), float(2*(1 - norm.cdf(abs(z))))
105
106 # — Bayesian LR via L2 with non-zero center (penalized IRLS) —————
107 class BayesianLogReg:
108     """Logistic regression with Gaussian priors  $\beta \sim N(\text{prior\_mean}, \text{prior\_sd}^2)$ .
109     Equivalent to L2 with non-zero center: minimize  $\|y - X\beta\|^2 + 0.5 \beta^T \Lambda \beta$ 
110     where  $\Lambda = \text{diag}(1/\text{prior\_sd}^2)$ . Solved by Newton-Raphson.
111     """
112     def __init__(self, prior_means, prior_sds, max_iter=200, tol=1e-6):
113         self.prior_means = np.asarray(prior_means, dtype=float)
114         self.prior_sds = np.asarray(prior_sds, dtype=float)
115         self.max_iter, self.tol = max_iter, tol
116     def fit(self, X, y):
117         X = np.asarray(X, dtype=float); y = np.asarray(y, dtype=float)
118         n, p = X.shape
119         Xa = np.column_stack([np.ones(n), X]) # add intercept
120         # Prior: weakly informative on intercept (sd=10), informative on coeffs
121         mu = np.concatenate([[0.0], self.prior_means])
122         sd = np.concatenate([[10.0], self.prior_sds])
123         Lambda = np.diag(1.0 / (sd ** 2))
124         beta = mu.copy()
125         for _ in range(self.max_iter):
126             eta = Xa @ beta
127             mu_p = 1.0 / (1.0 + np.exp(-np.clip(eta, -30, 30)))
128             W = mu_p * (1 - mu_p) + 1e-8
129             grad = Xa.T @ (mu_p - y) + Lambda @ (beta - mu)
130             H = (Xa.T * W) @ Xa + Lambda
131             step = np.linalg.solve(H, grad)
132             beta_new = beta - step
133             if np.max(np.abs(beta_new - beta)) < self.tol:
134                 beta = beta_new; break
135             beta = beta_new
136         self.coef_ = beta[1:].reshape(1, -1)
137         self.intercept_ = np.array([beta[0]])
138         return self
139     def predict_proba(self, X):
140         X = np.asarray(X, dtype=float)
141         eta = X @ self.coef_.ravel() + self.intercept_[0]
142         p = 1.0 / (1.0 + np.exp(-np.clip(eta, -30, 30)))
143         return np.column_stack([1 - p, p])
144
145
146 def derive_eicu_priors(features_bidmc):
147     """Fit elastic-net LR on eICU shared features, return prior means and sds
148     for BIDMC features (mapped). Unshared features get prior mean=0, sd=2.5.
149     """
150     print(" Fitting elastic-net LR on eICU to derive priors ...", flush=True)
151     eicu = load_eicu()
152     Xe = eicu[SHARED_EICU_NAMES].fillna(eicu[SHARED_EICU_NAMES].median()).astype(float)
153     ye = eicu["seizure"].astype(int)
154     sc = StandardScaler(); Xe_s = sc.fit_transform(Xe)
155     eicu_mu, eicu_sigma = sc.mean_, sc.scale_
156     lr = LogisticRegression(penalty="elasticnet", l1_ratio=0.5, C=0.1,
157                             solver="saga", max_iter=5000, class_weight="balanced",

```

```
158                                     n_jobs=1, random_state=SEED)
159     lr.fit(Xe_s, ye)
160     eicu_coefs = lr.coef_[0]
161 ... (170 more lines elided – see full script in the repository) ...
```

BayesianLogReg is a from-scratch implementation of logistic regression with Gaussian priors centred at user-supplied means. `derive_eicu_priors` fits an elastic-net LR on eICU’s shared features and returns those coefficients as the prior means.

Firth matches the BalancedRandomForest baseline on AUC (0.681 vs 0.676; DeLong $p = 0.81$) with a 3.3-fold improvement in Brier score. The Bayesian-with-eICU-priors variant degrades discrimination significantly — a biological-mismatch warning, not a statistical failure.

7 25_conformal_prediction.py — turning probabilities into bedside decisions

Conformal prediction turns any classifier into a procedure that emits prediction sets with distribution-free coverage guarantees. The class-conditional variant (Mondrian) guarantees coverage separately for each true class.

```
1  """Task 25 – Class-conditional (Mondrian) conformal prediction for clinical deployment.
2
3  Lit-review motivation: with n=655 / 48 events, AUC ceiling is essentially fixed
4  by Bernoulli noise. The decision-relevant reframing is calibrated risk
5  stratification with distribution-free coverage guarantees. Class-conditional
6  conformal prediction (Mondrian, Vovk 2003; modern: Angelopoulos & Bates 2021
7  arXiv:2107.07511; clinical: García-Cremades et al. PMLR 252, 2024) provides
8  exactly this.
9
10 We use the split-conformal scheme:
11 1. Train BalancedRF on the calibration-training split
12 2. On a held-out calibration set, compute nonconformity scores  $1 - P(\text{true class})$ 
13 3. Choose  $\alpha_*$  as the  $\alpha(1-)$  class-conditional quantile of those scores
14 4. For each test point, the prediction set is  $\{y : 1 - P(y) \leq \alpha_*\}$ 
15
16 Reported metrics:
17 • Empirical coverage (should  $\approx 1 - \alpha$ )
18 • Average prediction-set size
19 • Singleton-set rate (cases confidently classified)
20 • Rule-out rate at  $\alpha=0.10$  (cases assigned the "no seizure" singleton)
21 • Rule-in rate at  $\alpha=0.10$  (cases assigned the "seizure" singleton)
22
23 We compare three base models for the conformal procedure:
24 • BalancedRandomForest
25 • Diverse stack (from Task 22, if available)
26 • Bayesian LR (from Task 24, if available)
27
28 Outputs:
29 results/25_conformal.csv
30 figures/25_conformal.{png,pdf}
31 """
32 import sys, os
33 sys.path.insert(0, os.path.dirname(__file__))
34 os.environ["OMP_NUM_THREADS"] = "1"
35
36 import warnings; warnings.filterwarnings("ignore")
37 import numpy as np, pandas as pd
38 import matplotlib; matplotlib.use("Agg")
39 import matplotlib.pyplot as plt
40
41 from sklearn.model_selection import StratifiedKFold
42 from sklearn.metrics import roc_auc_score, brier_score_loss
43 from sklearn.pipeline import Pipeline
44 from sklearn.compose import ColumnTransformer
45 from sklearn.impute import SimpleImputer
46 from sklearn.preprocessing import StandardScaler
```

```

47 from imblearn.ensemble import BalancedRandomForestClassifier
48
49 from _shared import (load_bidmc, POSTOP_A_FEATURES, POSTOP_B_FEATURES,
50                      RES, FIG, SEED)
51
52 N_SPLITS = 5
53 N_REPEATS = 3
54 ALPHAS = [0.05, 0.10, 0.20]
55
56 def make_prep(features):
57     return ColumnTransformer([
58         ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
59                          ("sc", StandardScaler())]), features)]
60
61 def make_brf(features):
62     return Pipeline([("prep", make_prep(features)),
63                      ("clf", BalancedRandomForestClassifier(
64                          n_estimators=300, min_samples_leaf=2, n_jobs=1,
65                          random_state=SEED))])
66
67 def class_conditional_conformal(p_cal, y_cal, p_test, alpha):
68     """Class-conditional (Mondrian) split-conformal prediction.
69
70     For each class  $c$  separately, the nonconformity score is
71      $s_i = 1 - P(y_i = c \mid x_i)$ , evaluated on the calibration set. The
72      $-\alpha(1)(n_c+1)/n_c$  empirical quantile  $q_c$  (with finite-sample correction)
73     becomes the threshold. A test point's prediction set includes class  $c$ 
74     iff its nonconformity for  $c$  does not exceed  $q_c$ .
75
76     By construction the marginal coverage within each true class is at
77     least  $1 - \alpha$  – the Mondrian guarantee.
78
79     Parameters
80     -----
81     p_cal : array of  $P(y=1)$  on the calibration set.
82     y_cal : binary calibration outcomes.
83     p_test : array of  $P(y=1)$  on the test set.
84     alpha : target miscoverage in  $(0, 1)$ .
85
86     Returns
87     -----
88     np.ndarray of shape  $(n_{\text{test}}, 2)$  of booleans. Column 0 is whether class
89     'no seizure' ( $y=0$ ) is included in the prediction set; column 1 is
90     whether class 'seizure' ( $y=1$ ) is included.
91
92     References
93     -----
94     Vovk V, Gammerman A, Shafer G. Algorithmic Learning in a Random World.
95         Springer, 2005. (Foundational conformal-prediction reference.)
96     Angelopoulos AN, Bates S. A gentle introduction to conformal prediction.
97         arXiv:2107.07511, 2021. (Modern tutorial including Mondrian.)
98     García-Cremades S et al. Class-conditional conformal prediction for MACE
99         rule-out. PMLR 252, 2024. (Clinical application – rule-out fraction
100         as the headline metric, exactly as we use it here.)

```

```

101 """
102 p_cal = np.clip(p_cal, 1e-6, 1 - 1e-6)
103 p_test = np.clip(p_test, 1e-6, 1 - 1e-6)
104 # nonconformity = 1 - P(true class) on calibration set
105 nc_pos = 1.0 - p_cal[y_cal == 1] # cases where true=1
106 nc_neg = 1.0 - (1.0 - p_cal[y_cal == 0]) # cases where true=0 → 1-P(y=0)
107 if len(nc_pos) < 2 or len(nc_neg) < 2:
108     return np.zeros((len(p_test), 2), dtype=bool)
109 # finite-sample correction: (n+1)α(1-)/n th value
110 n_pos = len(nc_pos); n_neg = len(nc_neg)
111 q_pos = np.quantile(nc_pos, min(1.0, (1 - alpha) * (n_pos + 1) / n_pos))
112 q_neg = np.quantile(nc_neg, min(1.0, (1 - alpha) * (n_neg + 1) / n_neg))
113 # for each test point, include class c if its nonconformity ≤ q_c
114 include_pos = (1.0 - p_test) <= q_pos # include "1" if (1-P(1)) ≤ q_pos
115 include_neg = (1.0 - (1.0 - p_test)) <= q_neg # include "0" if (1-P(0)) ≤ q_neg
116 return np.column_stack([include_neg, include_pos]).astype(bool)
117
118
119 def evaluate_conformal(make_pipe_fn, features, X, y, alphas=ALPHAS,
120                       n_splits=N_SPLITS, n_repeats=N_REPEATS):
121     """Cross-conformal: split → fit on train, calibrate on val, predict on test.
122     We use a 3-way split per fold: train (60%), calibration (20%), test (20%).
123     Aggregate metrics over repeated CV.
124     """
125     metrics = {a: {"coverage": [], "set_size": [], "singleton_rate": [],
126                   "rule_out_rate": [], "rule_in_rate": []} for a in alphas}
127     rng = np.random.default_rng(SEED)
128     for rep in range(n_repeats):
129         skf = StratifiedKFold(n_splits=n_splits, shuffle=True,
130                               random_state=SEED + rep)
131         for fold_idx, (trainval, test) in enumerate(skf.split(X, y)):
132             # Split trainval → train (75%) + cal (25%)
133             yv = y.iloc[trainval]
134             pos_v = np.where(yv == 1)[0]; neg_v = np.where(yv == 0)[0]
135             rng_l = np.random.default_rng(SEED + rep * 100 + fold_idx)
136             rng_l.shuffle(pos_v); rng_l.shuffle(neg_v)
137             cal_pos = pos_v[: max(2, len(pos_v) // 4)]
138             cal_neg = neg_v[: max(2, len(neg_v) // 4)]
139             cal_idx = trainval[np.concatenate([cal_pos, cal_neg])]
140             train_idx = np.setdiff1d(trainval, cal_idx)
141     ... (132 more lines elided – see full script in the repository) ...

```

`class_conditional_conformal` computes the nonconformity score $1 - P(\text{true class})$ on every calibration example, takes the $(1-\alpha)(n+1)/n$ empirical quantile separately for the positive and negative classes, and for each test point includes each class in the prediction set iff its nonconformity does not exceed that class's quantile.

8 16_voi_evpi.py — value-of-information analysis

Value-of-information analysis answers: of all the uncertain parameters in our decision model, which would actually change the optimal strategy if we knew them perfectly? EVPI is the population-scale upper bound on the value of any future study. EVPPI ranks per-parameter research priorities.

```
1  """Task 16 – Value-of-information (EVPI / EVPPI) analysis for the CEA.
2
3  EVPI quantifies the maximum expected health-and-cost-equivalent value of
4  resolving all parameter uncertainty:
5
6      
$$\text{EVPI}(\lambda) = \theta E_{\lambda} [\max_d \text{NMB}(d, \theta; \lambda)] - \max_d \theta E_{\lambda} \text{NMB}(d, \theta; \lambda)$$

7
8   $\text{EVPPI}(\phi; \lambda)$  is the analogue restricted to a single parameter subset  $\phi$ , the
9  expected value of perfectly knowing  $\phi$ . We estimate it via the Strong-Oakley
10 (MDM 2014) non-parametric regression approach: a smooth fit of NMB on the
11 focal parameter, with the fitted values approximating  $E[\text{NMB} \mid \phi]$ .
12
13 WTP base case is $100,000/QALY (Neumann 2014; Vanness 2021; Crespo 2023).
14 We also report EVPI at $50k and $150k.
15
16 Outputs:
17     results/16_voi_evpi.csv
18     results/16_voi_evppi.csv
19     figures/16_voi_evpi.{png,pdf}
20 """
21 import sys, os
22 sys.path.insert(0, os.path.dirname(__file__))
23 os.environ["OMP_NUM_THREADS"] = "1"
24
25 import numpy as np, pandas as pd
26 import matplotlib; matplotlib.use("Agg")
27 import matplotlib.pyplot as plt
28 from scipy.interpolate import UnivariateSpline
29
30 # Reuse PSA helpers from the existing CEA module
31 from importlib.util import spec_from_file_location, module_from_spec
32 _HERE = os.path.dirname(__file__)
33 _spec = spec_from_file_location("cea_mod", os.path.join(_HERE, "10_11_cea_pairwise.py"))
34 cea = module_from_spec(_spec); _spec.loader.exec_module(cea)
35
36 from _shared import RES, FIG, SEED
37
38 # — Annual incident cSDH-surgery population (Move 3 lit input) —————
39 ANNUAL_POPULATION = 40_000      # midpoint of 30–50k cSDH operations / yr US
40 DECISION_HORIZON   = 10
41 DISCOUNT_RATE     = 0.03
42 WTPs                = (50_000, 100_000, 150_000)
43 N_PSA_VOI          = 5_000      # 5000 iterations – enough for stable EVPI
44
45 STRATEGIES = ["obs", "aed", "mla", "mlg"] # column suffixes in PSA frame
46
47
```

```

48 def run_psa_tracked(n=N_PSA_VOI, seed=SEED):
49     """Re-run the existing PSA but also persist the sampled parameters so
50     we can fit per-parameter EVPPi regressions."""
51     rng = np.random.default_rng(seed)
52     a_prev, b_prev = cea.beta_params_from_ci(0.12, 0.05, 0.20)
53     a_sens, b_sens = cea.beta_params_from_ci(0.842, 0.722, 0.918)
54     a_spec, b_spec = cea.beta_params_from_ci(0.504, 0.439, 0.569)
55     a_pse, b_pse = cea.beta_params_from_ci(0.10, 0.05, 0.18)
56     a_rrr, b_rrr = cea.beta_params_from_ci(0.45, 0.25, 0.65)
57     a_pe_det, b_pe_det = cea.beta_params_from_ci(0.03, 0.01, 0.06)
58     a_pe_und, b_pe_und = cea.beta_params_from_ci(0.12, 0.05, 0.22)
59     a_pdet_ceeg, b_pdet_ceeg = cea.beta_params_from_ci(0.95, 0.85, 0.99)
60     a_pdet_clin, b_pdet_clin = cea.beta_params_from_ci(0.30, 0.15, 0.50)
61     sh_aed_drug, sc_aed_drug = cea.gamma_params(200.0, 50)
62     sh_aed_adv, sc_aed_adv = cea.gamma_params(1507.0, 500)
63     sh_aed_cont, sc_aed_cont = cea.gamma_params(441.0, 150)
64     sh_aed_adv_c, sc_aed_adv_c = cea.gamma_params(1293.0, 400)
65     sh_aed_cont_c, sc_aed_cont_c = cea.gamma_params(140.0, 50)
66     sh_se_early, sc_se_early = cea.gamma_params(13200.0, 4000)
67     sh_se_late, sc_se_late = cea.gamma_params(35000.0, 10000)
68     sh_ceeg, sc_ceeg = cea.gamma_params(1500.0, 500)
69     sh_hosp, sc_hosp = cea.gamma_params(3500.0, 700)
70     sh_icu, sc_icu = cea.gamma_params(5000.0, 1500)
71     sh_szwu, sc_szwu = cea.gamma_params(2500.0, 800)
72     sh_epi, sc_epi = cea.gamma_params(15000.0, 5000)
73
74     rows = []
75     p_global = cea.params_global
76     for i in range(n):
77         ps = dict(
78             p_sz = rng.beta(a_prev, b_prev),
79             sens = rng.beta(a_sens, b_sens),
80             spec = rng.beta(a_spec, b_spec),
81             p_se = rng.beta(a_pse, b_pse),
82             rrr = rng.beta(a_rrr, b_rrr),
83             pe_d = rng.beta(a_pe_det, b_pe_det),
84             pe_u = rng.beta(a_pe_und, b_pe_und),
85             pd_ceeg = rng.beta(a_pdet_ceeg, b_pdet_ceeg),
86             pd_clin = rng.beta(a_pdet_clin, b_pdet_clin),
87             c_aed_d = rng.gamma(sh_aed_drug, sc_aed_drug),
88             c_aed_a = rng.gamma(sh_aed_adv, sc_aed_adv),
89             c_aed_c = rng.gamma(sh_aed_cont, sc_aed_cont),
90             aed_d = rng.uniform(14, 120),
91             c_aed_ac = rng.gamma(sh_aed_adv_c, sc_aed_adv_c),
92             c_aed_cc = rng.gamma(sh_aed_cont_c, sc_aed_cont_c),
93             aed_dc = rng.uniform(7, 42),
94             c_se_e = rng.gamma(sh_se_early, sc_se_early),
95             c_se_l = rng.gamma(sh_se_late, sc_se_late),
96             icu_red = rng.uniform(0.3, 1.5),
97             c_ceeg = rng.gamma(sh_ceeg, sc_ceeg),
98             c_hosp = rng.gamma(sh_hosp, sc_hosp),
99             c_icu = rng.gamma(sh_icu, sc_icu),
100            c_szwu = rng.gamma(sh_szwu, sc_szwu),
101            c_epi = rng.gamma(sh_epi, sc_epi),

```

```

102     u_dec     = rng.uniform(0.05, 0.20),
103     p_rec     = rng.beta(3, 17),
104     pd_b      = rng.beta(2.5, 97.5),
105     pd_e      = rng.beta(2, 98),
106     ceeg_d    = float(rng.choice([2, 3, 4, 5], p=[0.2, 0.4, 0.3, 0.1])),
107 )
108 shared = dict(
109     p_seizure_base=ps["p_sz"], aed_rrr=ps["rrr"],
110     cost_aed_drug=ps["c_aed_d"], cost_aed_adverse=ps["c_aed_a"],
111     cost_aed_continuation=ps["c_aed_c"], expected_aed_days=ps["aed_d"],
112     cost_aed_adverse_ceeg=ps["c_aed_ac"],
113     cost_aed_continuation_ceeg=ps["c_aed_cc"],
114     expected_aed_days_ceeg=ps["aed_dc"],
115     cost_se_early=ps["c_se_e"], cost_se_late=ps["c_se_l"],
116     icu_los_reduction=ps["icu_red"],
117     p_detect_ceeg=ps["pd_ceeg"], p_detect_clinical=ps["pd_clin"],
118     cost_ceeg=ps["c_ceeg"], cost_hosp_day=ps["c_hosp"], cost_icu=ps["c_icu"],
119     cost_sz_workup=ps["c_szwu"], p_se=ps["p_se"],
120     utility_sz_decrement=ps["u_dec"], p_recurrence=ps["p_rec"],
121     p_epilepsy_detected=ps["pe_d"], p_epilepsy_undetected=ps["pe_u"],
122     p_death_base=ps["pd_b"], p_death_excess=ps["pd_e"],
123     cost_annual_epi=ps["c_epi"], ceeg_days=ps["ceeg_d"],
124 )
125 ml_shared = dict(sens=ps["sens"], spec=ps["spec"], **shared)
126 c_o, q_o = cea.run_strategy("observation", p_global, **shared)
127 c_a, q_a = cea.run_strategy("universal_aed", p_global, **shared)
128 c_ma, q_ma = cea.run_strategy("ml_aed_only", p_global, **ml_shared)
129 c_mg, q_mg = cea.run_strategy("ml_guided", p_global, **ml_shared)
130 row = dict(ps)
131 row.update({
132     "cost_obs": c_o, "qaly_obs": q_o,
133     "cost_aed": c_a, "qaly_aed": q_a,
134     "cost_mla": c_ma, "qaly_mla": q_ma,
135     "cost_mlg": c_mg, "qaly_mlg": q_mg,
136 })
137 rows.append(row)
138 if (i + 1) % 500 == 0:
139     print(f"  PSA {i+1:>5d}/{n}", flush=True)
140 return pd.DataFrame(rows)
141 ... (134 more lines elided – see full script in the repository) ...

```

compute_evpi is the population-EVPI formula. evppi_strong_oakley implements the Strong-Oakley non-parametric regression approximation per parameter and feeds the tornado chart in Figure 6.

9 29_main_figures_jnnp.py — publication-grade figures

Every main paper figure is built by a dedicated function (figure_1 ...figure_6) with a consistent house style: Helvetica sans-serif, open-box axes, bold uppercase panel labels, a restrained CMYK-safe palette, 300-dpi PNG and editable vector PDF output.

```
1  """Task 29 – Main paper figures, JNNP-style.
2
3  JNNP aesthetic conventions (consistent with the journal's published figures):
4      • Helvetica / Arial sans-serif throughout (DejaVu Sans fallback)
5      • Open-box axes: only bottom and left spines visible; tick direction out
6      • Bold uppercase panel labels (A, B, C) in the top-left of each panel
7      • Restrained palette: navy / rust / forest-green / muted-grey
8      • Light grey y-axis grid only, alpha 0.30
9      • 300 dpi PNG (print) and vector PDF (for figure submission)
10     • CMYK-safe primary colours; usable in grayscale conversion
11
12 Rebuilds –F1F6 as native matplotlib figures (no PIL image-wrapping),
13 sourced directly from the result CSVs and the existing scripts' data.
14
15 Outputs:
16     figures/F1_discrimination.{png,pdf}
17     figures/F2_calibration_dca.{png,pdf}
18     figures/F3_method_battery.{png,pdf}
19     figures/F4_conformal.{png,pdf}
20     figures/F5_cea.{png,pdf}
21     figures/F6_voi.{png,pdf}
22 """
23 import sys, os
24 sys.path.insert(0, os.path.dirname(__file__))
25 os.environ["OMP_NUM_THREADS"] = "1"
26
27 import numpy as np, pandas as pd
28 import matplotlib; matplotlib.use("Agg")
29 import matplotlib.pyplot as plt
30 from matplotlib import font_manager
31 import matplotlib.patches as mpatches
32 import json
33
34 from _shared import RES, FIG, CACHE
35
36 # — JNNP style —————
37 # Prefer Helvetica / Arial; fall back to a sans-serif that ships with matplotlib.
38 _pref_fonts = ["Helvetica", "Helvetica Neue", "Arial", "Liberation Sans",
39               "DejaVu Sans"]
40 _available = {f.name for f in font_manager.fontManager.ttflist}
41 JNNP_FONT = next((f for f in _pref_fonts if f in _available), "DejaVu Sans")
42
43 plt.rcParams.update({
44     "font.family": "sans-serif",
45     "font.sans-serif": [JNNP_FONT],
46     "font.size": 9,
47     "axes.titlesize": 10,
```

```

48     "axes.titleweight": "bold",
49     "axes.labelsize": 9,
50     "axes.labelweight": "regular",
51     "axes.edgecolor": "#222222",
52     "axes.linewidth": 0.8,
53     "axes.spines.top": False,
54     "axes.spines.right": False,
55     "axes.grid": True,
56     "axes.axisbelow": True,
57     "grid.color": "#cccccc",
58     "grid.linewidth": 0.5,
59     "grid.alpha": 0.45,
60     "xtick.direction": "out",
61     "ytick.direction": "out",
62     "xtick.major.size": 4,
63     "ytick.major.size": 4,
64     "xtick.minor.size": 2,
65     "ytick.minor.size": 2,
66     "xtick.major.width": 0.7,
67     "ytick.major.width": 0.7,
68     "xtick.color": "#222222",
69     "ytick.color": "#222222",
70     "xtick.labelsize": 8,
71     "ytick.labelsize": 8,
72     "legend.fontsize": 8,
73     "legend.frameon": False,
74     "legend.handlelength": 1.5,
75     "legend.handletextpad": 0.6,
76     "figure.dpi": 110,
77     "savefig.dpi": 300,
78     "savefig.bbox": "tight",
79     "savefig.pad_inches": 0.05,
80     "pdf.fonttype": 42,          # editable text in PDF
81     "ps.fonttype": 42,
82 })
83
84 # JNNP-safe palette (CMYK-friendly, distinct in grayscale)
85 COL = {
86     "navy":    "#1F3D5C",
87     "rust":    "#B5532C",
88     "forest":  "#2E6B45",
89     "indigo":  "#5C4B8A",
90     "ochre":   "#B58A2E",
91     "slate":   "#4A5560",
92     "soft":    "#9FB6C9",
93     "rule":    "#000000",
94     "grey":    "#6E6E6E",
95     "highlight": "#D03C29",
96 }
97
98 def add_panel_label(ax, label, *, offset=(-0.12, 1.04)):
99     ax.text(offset[0], offset[1], label,
100             transform=ax.transAxes,
101             fontsize=11, fontweight="bold",

```

```

102         va="bottom", ha="left")
103
104     def style_axis(ax, *, ygrid=True, xgrid=False):
105         ax.tick_params(axis="both", which="major")
106         if ygrid:
107             ax.yaxis.grid(True, color=COL["soft"], alpha=0.30, linewidth=0.4)
108         if xgrid:
109             ax.xaxis.grid(True, color=COL["soft"], alpha=0.30, linewidth=0.4)
110         else:
111             ... (512 more lines elided – see full script in the repository) ...

```

The `plt.rcParams.update` block sets the house style once; every panel inherits from it. `add_panel_label` puts the bold A/B/C labels; `style_axis` applies the tick direction; `figure_legend_below` places a single shared legend beneath the figure so legends never overlap data.

10 27_build_jnnp_manuscript.py — building the Word manuscript from code

This script writes the manuscript .docx file end-to-end using python-docx. Building the manuscript programmatically means every numerical claim in the text is traceable to the results CSV that produced it, and every version of the manuscript is a deterministic function of the underlying analysis.

```
1  """Task 27 – Journal-format manuscript build (proof-of-concept framing).
2
3  Target: a high-impact neurology/neurosurgery journal accepting Original
4  Research articles with the following conventional limits (most BMJ-family
5  and similar journals):
6
7      • Original Research: 4,000 words main text
8      • Structured abstract: 250 words
9      • Maximum 6 figures/tables in main text
10     • Vancouver references (numbered, superscript)
11     • TRIPOD-AI reporting checklist for prediction models
12     • Supplementary material accepted separately
13
14  Outputs:
15     manuscript/main_manuscript.docx    ≤(4000 words, –F1F6, Table 1)
16     manuscript/supplementary.docx      (Figures –S1S7, Tables –S1S5,
17                                         TRIPOD-AI checklist)
18
19  """
20  import sys, os
21  sys.path.insert(0, os.path.dirname(__file__))
22  import pandas as pd
23
24  from docx import Document
25  from docx.shared import Inches, Pt, RGBColor
26  from docx.enum.text import WD_ALIGN_PARAGRAPH, WD_LINE_SPACING
27  from docx.xml.ns import qn
28  from docx.xml import XmlElement
29
30  from _shared import OUT, RES, FIG
31
32  MANUS_DIR = OUT / "manuscript"
33  MAIN_PATH = MANUS_DIR / "main_manuscript.docx"
34  SUPP_PATH = MANUS_DIR / "supplementary.docx"
35
36  # — Doc helpers —————
37  def add_heading(doc, text, level=1):
38      h = doc.add_heading(text, level=level)
39      for r in h.runs:
40          r.font.color.rgb = RGBColor(0x14, 0x1F, 0x3A)
41          r.font.name = "Times New Roman"
42      return h
43
44  def add_para(doc, text, *, italic=False, bold=False, indent=False, size=11,
45              alignment=WD_ALIGN_PARAGRAPH.JUSTIFY):
46      p = doc.add_paragraph()
47      p.alignment = alignment
48      p.paragraph_format.line_spacing_rule = WD_LINE_SPACING.ONE_POINT_FIVE
```

```

47     p.paragraph_format.space_after = Pt(6)
48     if indent: p.paragraph_format.first_line_indent = Inches(0.3)
49     r = p.add_run(text)
50     r.font.size = Pt(size); r.font.name = "Times New Roman"
51     r.italic = italic; r.bold = bold
52     return p
53
54 def add_runs(doc, runs, *, indent=True, alignment=WD_ALIGN_PARAGRAPH.JUSTIFY):
55     """runs = list of (text, dict-of-formatting)."""
56     p = doc.add_paragraph()
57     p.alignment = alignment
58     p.paragraph_format.line_spacing_rule = WD_LINE_SPACING.ONE_POINT_FIVE
59     p.paragraph_format.space_after = Pt(6)
60     if indent: p.paragraph_format.first_line_indent = Inches(0.3)
61     for text, fmt in runs:
62         r = p.add_run(text)
63         r.font.size = Pt(fmt.get("size", 11)); r.font.name = "Times New Roman"
64         r.bold = fmt.get("bold", False)
65         r.italic = fmt.get("italic", False)
66         if fmt.get("superscript"):
67             r.font.superscript = True
68     return p
69
70 def add_figure(doc, path, *, caption, width=Inches(6.5)):
71     if not os.path.exists(path):
72         add_para(doc, f"[Figure missing: {path}]", italic=True); return
73     p = doc.add_paragraph(); p.alignment = WD_ALIGN_PARAGRAPH.CENTER
74     p.add_run().add_picture(str(path), width=width)
75     cp = doc.add_paragraph(); cp.alignment = WD_ALIGN_PARAGRAPH.LEFT
76     cr = cp.add_run(caption)
77     cr.font.size = Pt(10); cr.italic = True; cr.font.name = "Times New Roman"
78
79 def add_table_from_df(doc, df, caption=None):
80     if caption:
81 ... (751 more lines elided – see full script in the repository) ...

```

The structure mirrors a normal Word workflow: `setup_document` sets page margins; `add_heading`, `add_para`, `add_runs` handle text; `register_figure` and `register_table` accumulate figures and tables; `render_collected_tables_and_figures` emits them all in a single Tables-and-Figures section at the end.

11 30_export_calculator_assets.py — packaging the model for the dashboard

The browser-side calculator cannot run scikit-learn — it is JavaScript. To let the browser compute the same prediction the Python model would, this script extracts the deployment-ready coefficients, the feature scaler statistics, and the conformal quantiles and writes them to site/model_assets.json.

```
1  """Task 30 – Export deployment-ready assets for the web calculator.
2
3  Produces:
4      github_repo/site/model_assets.json
5      • Firth penalized LR coefficients fit on the FULL BIDMC cohort
6      • Feature means / SDs for per-feature z-scaling
7      • Class-conditional conformal nonconformity quantiles at  $\alpha \in \{0.05, 0.10, 0.20\}$ 
8      • CEA base-case parameters (cost / QALY per strategy)
9      • Population-savings template (per-patient  $\Delta$ Cost,  $\Delta$ QALY versus
10         observation for ML-AED and ML-cEEG)
11
12  The web calculator at site/calculator.html consumes this JSON to produce
13  a deterministic prediction without any patient data leaving the browser.
14  """
15  import sys, os, json
16  sys.path.insert(0, os.path.dirname(__file__))
17  os.environ["OMP_NUM_THREADS"] = "1"
18
19  import warnings; warnings.filterwarnings("ignore")
20  import numpy as np, pandas as pd
21
22  from sklearn.impute import SimpleImputer
23  from sklearn.preprocessing import StandardScaler
24  from sklearn.model_selection import StratifiedKFold
25  from firthlogist import FirthLogisticRegression
26
27  from _shared import load_bidmc, POSTOP_A_FEATURES, OUT, SEED
28
29  SITE_DIR = OUT / "github_repo" / "site"
30  SITE_DIR.mkdir(parents=True, exist_ok=True)
31
32
33  def main():
34      df = load_bidmc()
35      y = df["seizure"].astype(int).values
36      X_raw = df[POSTOP_A_FEATURES].copy()
37
38      imp = SimpleImputer(strategy="median")
39      X_imp = pd.DataFrame(imp.fit_transform(X_raw), columns=POSTOP_A_FEATURES)
40      sc = StandardScaler()
41      X_z = sc.fit_transform(X_imp)
42
43      # Firth on full data – for deployment coefficients
44      print(f"Fitting Firth penalized LR on full BIDMC (n={len(y)}, events={int(y.sum())})...",
45            flush=True)
46      firth = FirthLogisticRegression(max_iter=200, max_halfstep=25, tol=1e-6,
```

```

47 skip_pvals=True, skip_ci=True, wald=True)
48 firth.fit(X_z, y)
49 p_full = firth.predict_proba(X_z)[: , 1]
50 intercept = float(firth.intercept_)
51 coefs = firth.coef_.tolist()
52 print(f" intercept = {intercept:.4f}")
53 for f, c in zip(POSTOP_A_FEATURES, coefs):
54     print(f"  β[{f:<22s}] = {c:+.4f}")
55
56 # — Class-conditional conformal quantiles —
57 # Use the same split as the conformal evaluation (5-fold, fit on 75% +
58 # calibrate on 25%) and aggregate the nonconformity scores
59 print("\nComputing class-conditional conformal nonconformity quantiles...",
60       flush=True)
61 nc_pos_all, nc_neg_all = [], []
62 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
63 for tr, te in skf.split(X_z, y):
64     # split trainval -> train (75%) + cal (25%)
65     rng = np.random.default_rng(SEED)
66     pos_tr = np.where(y[tr] == 1)[0]; neg_tr = np.where(y[tr] == 0)[0]
67     rng.shuffle(pos_tr); rng.shuffle(neg_tr)
68     cal_pos = tr[pos_tr[: max(2, len(pos_tr) // 4)]]
69     cal_neg = tr[neg_tr[: max(2, len(neg_tr) // 4)]]
70     cal_idx = np.concatenate([cal_pos, cal_neg])
71     train_idx = np.setdiff1d(tr, cal_idx)
72     if y[train_idx].sum() < 5: continue
73     m = FirthLogisticRegression(max_iter=200, max_halfstep=25, tol=1e-6,
74                                skip_pvals=True, skip_ci=True, wald=True)
75     m.fit(X_z[train_idx], y[train_idx])
76     p_cal = np.clip(m.predict_proba(X_z[cal_idx])[:, 1], 1e-6, 1 - 1e-6)
77     y_cal = y[cal_idx]
78     nc_pos = (1.0 - p_cal[y_cal == 1])
79     nc_neg = (1.0 - (1.0 - p_cal[y_cal == 0]))
80     nc_pos_all.extend(nc_pos.tolist())
81     nc_neg_all.extend(nc_neg.tolist())
82 nc_pos_all = np.array(nc_pos_all); nc_neg_all = np.array(nc_neg_all)
83 alphas = [0.05, 0.10, 0.20]
84 q_pos = {f"alpha_{a:.2f}": float(np.quantile(nc_pos_all, 1 - a)) for a in alphas}
85 q_neg = {f"alpha_{a:.2f}": float(np.quantile(nc_neg_all, 1 - a)) for a in alphas}
86 print(f" positive-class nonconformity quantiles: {q_pos}")
87 print(f" negative-class nonconformity quantiles: {q_neg}")
88
89 # — CEA base-case (per-patient expected outcomes) —
90 # values from scripts/14_decision_tree.py rollback
91 cea_rollback = pd.read_csv(OUT / "results" / "14_decision_tree_rollback.csv")
92 cea_dict = cea_rollback.set_index("strategy")[["E_cost_USD", "E_QALY"]].to_dict()
93
94 # — Package and write —
95 payload = {
96     "model": {
97         "type": "Firth penalized logistic regression",
98         "fit_on": "BIDMC postoperative-A, n=655, 48 events",
99         "intercept": intercept,
100        "features": POSTOP_A_FEATURES,

```

```

101     "coefficients": coefs,
102     "feature_means": sc.mean_.tolist(),
103     "feature_sds":    sc.scale_.tolist(),
104     "feature_medians_for_imputation": imp.statistics_.tolist(),
105 },
106 "conformal": {
107     "method": "class-conditional (Mondrian) split conformal",
108     "calibration_split": "75% train / 25% calibration, 5-fold pooled",
109     "nonconformity_quantiles_positive": q_pos,
110     "nonconformity_quantiles_negative": q_neg,
111 },
112 "cea": {
113     "wtp_threshold_usd_per_qaly": 100000,
114     "discount_rate": 0.03,
115     "horizon_years": 10,
116     "strategies": cea_dict,
117 },
118 "metadata": {
119     "n_features": len(POSTOP_A_FEATURES),
120     "n_patients": int(len(y)),
121 ... (14 more lines elided – see full script in the repository) ...

```

The Firth model is fit on the full BIDMC cohort. The class-conditional conformal quantiles are computed by repeated 5-fold splits with 75/25 train/calibration partitioning; the $(1-\alpha)(n+1)/n$ quantile of each class is taken. Everything is bundled into a single JSON file.

12 The interactive dashboards — HTML, CSS, JavaScript

The three dashboards on the companion site are static HTML pages served by GitHub Pages. Each page bundles its own JavaScript so all computation happens in the user's browser.

12.1 Anatomy of an HTML page

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Risk calculator</title>
6   <style> /* CSS – visual styling */ </style>
7 </head>
8 <body>
9   <header>...</header>
10  <main>...</main>
11  <script> /* JavaScript – behaviour */ </script>
12</body>
13</html>
```

HTML defines structure; CSS defines visual style; JavaScript defines behaviour.

12.2 How the risk calculator works

On page load the calculator fetches `model_assets.json`, extracts the Firth coefficients and the conformal quantiles, and attaches an event listener to every form input. Whenever the user changes any input, the `recompute` function fires: it reads all 21 feature values, z-scales them using the means and SDs from the JSON, computes the dot product with the coefficients, adds the intercept, applies the sigmoid, and produces a probability.

```
1 function recompute() {
2   if (!MODEL) return;
3   const raw = FEATURES.map(f => val(f));
4   const M = MODEL.model;
5   let z = 0;
6   for (let i = 0; i < raw.length; i++) {
7     z += ((raw[i] - M.feature_means[i]) / M.feature_sds[i]) * M.coefficients[i];
8   }
9   z += M.intercept;
10  const p = 1 / (1 + Math.exp(-z));
11  document.getElementById("kpi-prob").textContent = fmtPct(p);
12  document.getElementById("kpi-prob-ratio").textContent =
13    (p / MODEL.metadata.prevalence).toFixed(2) + "x prevalence";
14  document.getElementById("kpi-prob-meta").textContent =
15    "Logit = " + z.toFixed(3) + " · base rate 7.3%";
16
17  const alpha = parseFloat(document.getElementById("alpha").value);
18  const aKey = "alpha_" + alpha.toFixed(2);
19  const qPos = MODEL.conformal.nonconformity_quantiles_positive[aKey];
20  const qNeg = MODEL.conformal.nonconformity_quantiles_negative[aKey];
```

```

21 const incPos = (1 - p) <= qPos;
22 const incNeg = p <= qNeg;
23
24 let label, recText, palette;
25 if (incPos && !incNeg) {
26   label = "{ seizure }"; palette = "rust";
27   recText = "RULE-IN. At the chosen confidence level the patient is high-risk. Consider AED
28     prophylaxis and/or continuous EEG monitoring.";
29 } else if (!incPos && incNeg) {
30   label = "{ no seizure }"; palette = "forest";
31   recText = "RULE-OUT. At the chosen confidence level postoperative seizure can be
32     confidently ruled out. AED prophylaxis may be safely omitted with routine observation.";
33 } else if (incPos && incNeg) {
34   label = "{ seizure, no seizure }"; palette = "ochre";
35   recText = "INDETERMINATE. The model cannot give a confident singleton prediction at this  $\alpha$ .
36     Clinical judgment should guide the AED-vs-monitoring decision.";
37 } else {
38   label = "{  $\emptyset$  }"; palette = "ochre";
39   recText = "NULL SET. Neither class is supported at this  $\alpha$  – interpret as an outlier
40     relative to the calibration set.";
41 }
42 document.getElementById("kpi-set").textContent = label;
43 document.getElementById("kpi-set-meta").textContent =
44   " $\alpha$  = " + alpha.toFixed(2) + "  $\rightarrow$  " + ((1 - alpha) * 100).toFixed(0) + "% coverage";
45
46 const recCard = document.getElementById("rec-card");
47 recCard.classList.remove("bg-ochre-50", "bg-rust-50", "bg-forest-50",
48   "dark:bg-ochre-900/20", "dark:bg-rust-900/20", "dark:bg-forest-900/20",
49   "ring-ochre-700/30", "ring-rust-700/30", "ring-forest-700/30");
50 recCard.classList.add(`bg-${palette}-50`, `dark:bg-${palette}-900/20`,
51   `ring-${palette}-700/30`);
52 const dt = recCard.querySelector("dt");
53 dt.classList.remove("text-ochre-700", "text-rust-700", "text-forest-700",
54   "dark:text-ochre-500", "dark:text-rust-500", "dark:text-forest-500");
55 dt.classList.add(`text-${palette}-700`, `dark:text-${palette}-500`);
56
57 document.getElementById("rec-text").textContent = recText;
58 document.getElementById("alpha-out").textContent = " $\alpha$  = " + alpha.toFixed(2);
59 document.getElementById("alpha-meta").textContent =
60   ((1 - alpha) * 100).toFixed(0) + "% coverage guarantee.";
61 ... (25 more lines elided – see full script in the repository) ...

```

The dot product is a single for loop. Each iteration takes one raw feature value, z-scales it, multiplies by the corresponding Firth coefficient, and accumulates into z. After the loop, the intercept is added and the sigmoid applied: $p = 1/(1+\exp(-z))$. That gives the predicted probability. The conformal logic that follows checks whether $1-p$ is below each class-conditional quantile; the answer determines the prediction set and the recommendation.

12.3 How the savings calculator works

The savings calculator's JavaScript multiplies the per-patient ΔCost and ΔQALY values by the user-specified cohort size, applies the chosen discount rate over the chosen horizon, and presents the result

as KPI cards and a net-monetary-benefit bar chart. No external libraries are used — the bars are styled `<div>` elements whose width is set proportionally.

12.4 How the interactive callgraph works

The callgraph dashboard uses one external library — `vis-network` — loaded from a CDN. On page load the script fetches `callgraph.json`, maps each script to a node with category-coloured fill, attaches the per-node function inventory as a hidden property, and renders the network with a force-directed layout.

13 The callgraph — how the scripts depend on each other

Every analysis script in the repository imports `_shared.py`. Many also import each other. The callgraph is generated by parsing each `.py` file's Abstract Syntax Tree (Python's machine-readable representation of its own source) and extracting the import statements and top-level function definitions.

Before you modify any function in `_shared.py`, look at the callgraph and ask yourself: which scripts import this function? Any script with an arrow pointing into `_shared.py` depends on its behaviour. Test downstream before publishing the change.

14 Other scripts — what they do, in one paragraph each

14.1 03_dca.py

Decision-curve analysis. Reads cached out-of-fold predictions, computes Vickers net benefit at every probability threshold from 0 to 30 percent, plots the model curve against treat-all and treat-none, and writes the summary CSV used by Figure 2 panel B.

14.2 05_temporal_leakage.py

Temporal-leakage audit. Excludes the 24-hour and 48-hour rolling lab/vital features and the prophylactic-AED indicator, refits, and reports the resulting AUC.

14.3 06_overfitting.py

Overfitting diagnostics. Repeats the BalancedRandomForest fit with nested cross-validation, compares variable-importance ranks across folds (Spearman rho across 25 folds = 0.98), and produces a learning-curve plot.

14.4 07_missing_data.py

Missing-data sensitivity. Runs Little’s MCAR test, computes the missingness-versus-outcome chi-square per feature, and pools AUC estimates across ten multiply-imputed datasets via Rubin’s rules.

14.5 08_eicu_cohort.py

eICU cohort definition sensitivity. Fits the model in each of four prespecified cohort strata and reports bootstrap 95% AUC CIs.

14.6 09_competing_risks.py

Cause-specific Cox model plus IPCW Fine-Gray subdistribution-hazard model. Reports concordance indices and the Grambsch-Therneau Schoenfeld test.

14.7 10_11_cea_pairwise.py

Cost-effectiveness analysis with 10,000-iteration PSA. Defines the Params dataclass; samples each parameter from its own beta or gamma distribution; runs each of the four strategies; reports the cost-effectiveness plane and the acceptability curve.

14.8 12_nis_seizure_reclassify.py

The NIS outcome-correction analysis. Distinguishes acute symptomatic seizure from pre-existing epilepsy and reports both definitions’ AUC.

14.9 13_nis_grouped_lasso.py

Group-LASSO with lambda-path tuning on the NIS chronic+surgical cohort under the corrected outcome.

14.10 15_radiology_nlp.py

Regex-pattern radiology NLP extractor with a synthetic validation set. Released as a tool; not promoted to a paper contribution.

14.11 17_build_slides.py

Builds the 15-minute oral-presentation deck using python-pptx.

14.12 18_bidmc_optimize.py

BIDMC model-optimization sweep. Optuna for 40 trials each on XGBoost and LightGBM; stacking ensemble; DeLong tests against the baseline.

14.13 19_transfer_learning.py

eICU to BIDMC transfer-learning experiment. Trains an XGBoost on the seven shared features, scores BIDMC patients, and uses the score as an additional feature in the BIDMC model.

14.14 20_build_manuscript.py

An earlier manuscript-build script. Superseded by 27.

14.15 21_imbalance_sweep.py

Eleven-method class-imbalance sweep on BIDMC.

14.16 22_diverse_stacking.py

Diverse-base stacking ensemble (LR + BalancedRF + XGBoost + KNN + RBF-SVM with a logistic meta-learner).

14.17 23_tabpfn_eval.py

Attempted TabPFN v2 evaluation. Reports the API-credential requirement and falls back.

14.18 26_main_figures.py

Earlier figure-builder that composes existing PNGs via PIL. Superseded by 29.

14.19 28_make_callgraph.py

Parses every .py file via ast, emits Markdown + Mermaid callgraph.

14.20 31_export_callgraph_json.py

Same parsing pipeline as 28, emits JSON for the interactive callgraph dashboard.

14.21 32_build_code_companion.py

Builds the Word version of this document.

14.22 33_build_code_companion_pdf.py

This script — the one that built the PDF you are reading.

15 Further study — Claude skills that help you learn the code

Claude ships with a library of task-focused expert prompts called skills, each invoked by typing the skill name with a leading slash. The following skills are particularly relevant to studying this codebase.

`/claude-code-guide`

General Python idioms, scikit-learn patterns, the Anthropic API.

`/code-review-excellence`

Structured second-pair-of-eyes review with prioritised feedback.

`/documentation-writer`

Docstrings, README sections, function-level comments (Diataxis).

`/python-project-structure`

Why a Python project is organised the way it is.

`/scientific-writing`

Translating analysis results into manuscript prose.

`/scientific-critical-thinking`

Interrogating methodological choices and bias diagnostics.

`/visualization-best-practices`

Refining figures, colour-blind safety, annotation placement.

`/statistical-analysis`

Choosing statistical tests, assumption checking, reporting.

`/the-humanizer`

Removing AI-pattern phrasing from a draft.

`/peer-review`

Structured manuscript review applying CONSORT or STROBE or TRIPOD.

`/literature-review`

Comprehensive systematically-searched literature review.

/exploratory-data-analysis

Initial inspection of a new dataset with quality metrics.

/scikit-learn

Specific questions about scikit-learn pipelines and CV.

/statsmodels

Formal frequentist inference beyond scikit-learn.

/pymc

Fitting Bayesian models with MCMC.

/improve-codebase-architecture

Refactoring after reading; reducing duplication.

(1) Read Chapter 1 of this document to learn the Python constructs you will see. (2) Read `_shared.py` with Chapter 2 of this document open beside you. (3) Read `02_calibration.py` and `24_firth_bayes_lr.py` with Chapters 3 and 6 open. (4) Open the calculator dashboard in your browser, then come back to Chapter 12 to see how the JavaScript implements the same model. By that point the rest of the repository will feel familiar.